DATA STRUCTURES AND ALGORITHMS

Lecture Notes 1

Pradit Pitaksathienkul

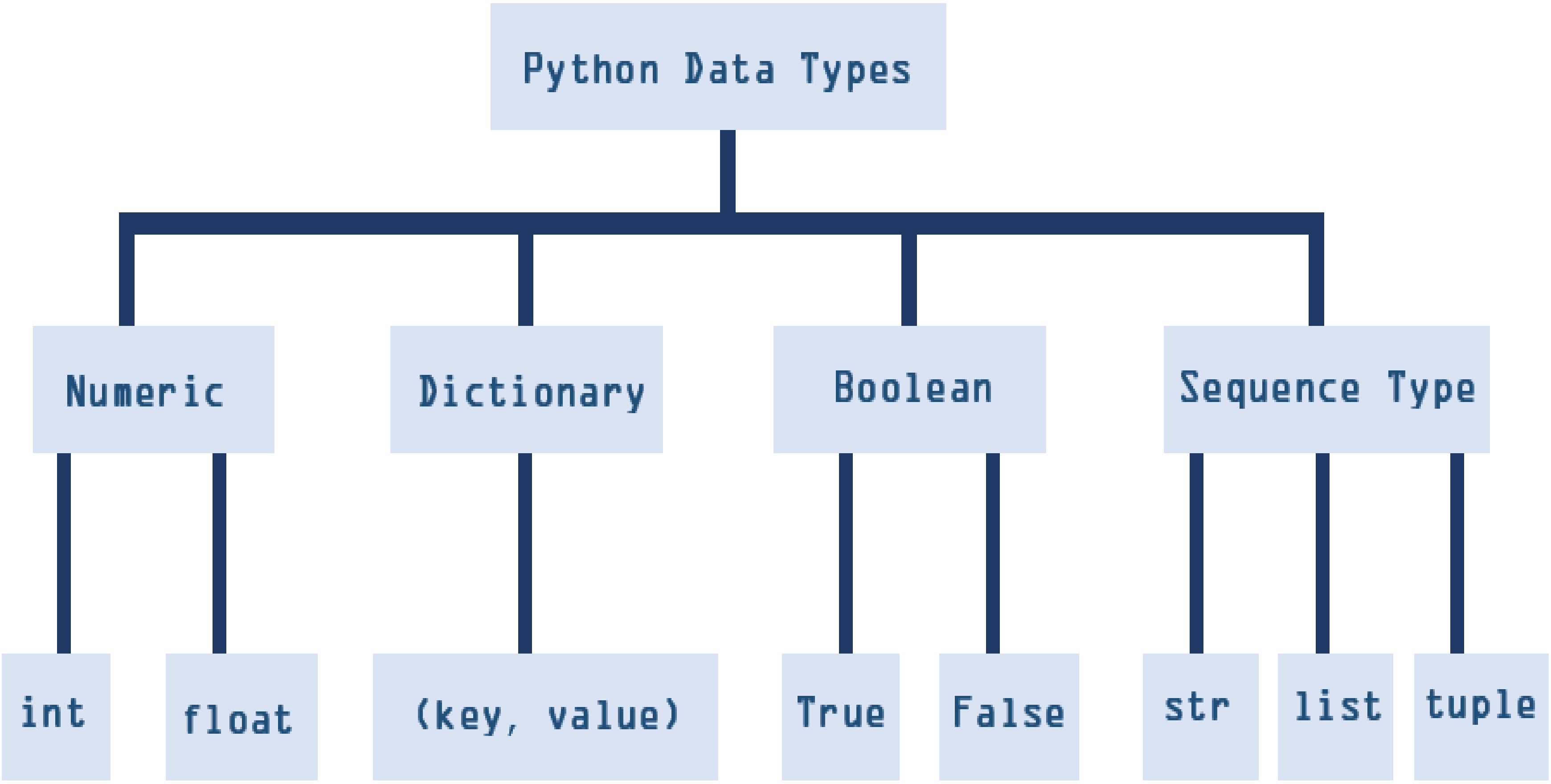
ROAD MAP

- **Data Types**
- Abstract Data Types
- Algorithms
- Data Structures
- Time & Space Complexity

Data Types

- An attribute of data that informs the interpreter on how to classify the variable.
  - Python there are four categories of data types: numeric, sequential, booleans, and dictionary

Python Data Types



Abstract Data Types (ADT)

- A logical process on how we view data and allowed operations without the regard on how it will be implemented.

  (Mathematical abstraction)
- Think of an abstract data type as a process of the rules and operations of data.
  - There is no concrete implementation.

Algorithms

- A set of **step-by-step instructions** used to solved a problem.
- One example of an algorithm can be a recipe from a cookbook.
- When we follow **step-by-step instructions** from the cook book to create our meal.

Algorithms

- For an example "how to make omelet":
- 1. Crack the eggs into a bowl.
- 2. Melt the butter over medium-low heat, and keep the temperature low and slow when cooking the eggs.
- 3. Add the eggs to the skillet and cook without stirring until the edges begin to set.
- 4. Add the filling when the eggs begin to set. Cook for a few more seconds.
- 5. Fold the omelet in half. Slide it onto a plate.

Data Structures

- A data structure is a format for accessing, storing, organizing, or structuring data such as array, linked list, stack, queue, tree, graph, hash table.
- The implementation of an abstract data type can be referred to a data structure

Data Structures and Abstract Data Types

- ADT gives us the blue print

while a data structure tells us how to implement it.

ADT = ประเภทข้อมูลนามธรรม คือ พิมพ์เขียว

data structure = ตัวที่บอกเราว่าจะนำพิมพ์เขียวนั้นไปสร้าง หรือใช้งานจริงได้อย่างไร

Data Structures and Algorithms

- An algorithm processes data and that data is then stored into a data structure.

```
data = [0, 1, -1, 5]
```

← Data Structure

```
order = data.sort()
```

Algorithm

Time & Space Complexity

- Often, there is more than one way to solve the same problem with different programs. So **how would you be able to compare the performance of different algorithms**, is one program better than the other?

Time & Space Complexity

- There are two ways to determine which algorithm is more efficient:

    • The amount of space or memory an algorithm requires

    • The amount of time an algorithm requires to execute

- Overall time and space complexity can be impacted from several factors such as hardware, operating system.

Time & Space Complexity

- If we had to choose between either optimizing time or optimizing space.
- Overall, it depends on your needs, but in a production setting, **optimizing time is the main priority** because we can buy memory, but we can't buy time!
- This can lead to the best trade off = จุดลงตัว of both increasing the space and lowering the time!